ECA5801-IOE AND AI DRIVEN SYSTEM

SIMULATION

EXPERIMENT 11

K.POOJA SREE

192512245

Simulation and Performance Analysis of Big Data Processing for IoE Sensor Data using Hadoop and Apache Spark.

```
# ============================================================
# SIMULATION EXPERIMENT - 3
# Big Data Processing for IoE Sensor Data using Hadoop and Spark
# Google Colab - Complete Corrected Single-Cell Code
# ============================================================

import os
import time
import gc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

# ---------------------- CONFIGURATION ----------------------------

NUM_DEVICES = 100
DATASET_SIZES = [1_000_000, 5_000_000, 10_000_000]
NUM_RUNS = 3
CHUNK_SIZE = 200_000
```

```
TEMP_THRESHOLD = 40.0

OUTPUT_DIR = "/content/ioe_big_data_results"


os.makedirs(OUTPUT_DIR, exist_ok=True)


COLUMNS = [

   "device_id", "timestamp", "temperature",

   "humidity", "vibration"

]


rng = np.random.default_rng(42)


print("=" * 70)

print("IoE BIG DATA PROCESSING: HADOOP-STYLE vs SPARK-STYLE
SIMULATION")

print("=" * 70)

print(f"Devices: {NUM_DEVICES}")

print(f"Dataset sizes: {DATASET_SIZES}")

print(f"Runs per method: {NUM_RUNS}")

print("Generating datasets in chunks to reduce memory usage...\n")




# ---------------------- DATA GENERATION -------------------------


def generate_dataset(n_records, path):

   """Generate IoE sensor records in chunks and save them as CSV."""
```

```
if os.path.exists(path):
    os.remove(path)

start_epoch = 1_760_000_000
written = 0
expected_temp_sum = np.zeros(NUM_DEVICES, dtype=np.float64)
expected_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
expected_hot_counts = np.zeros(NUM_DEVICES, dtype=np.int64)

first_chunk = True

while written < n_records:
    n = min(CHUNK_SIZE, n_records - written)

    device_ids = rng.integers(
        0, NUM_DEVICES, size=n, dtype=np.int32
    )
    timestamps = start_epoch + np.arange(
        written, written + n, dtype=np.int64
    )
    temperatures = np.round(
        rng.normal(30, 8, size=n), 2
    )
    humidity = np.round(
        rng.uniform(25, 90, size=n), 2
```

```
)
vibration = np.round(
    rng.uniform(0, 5, size=n), 3
)

chunk = pd.DataFrame({
    "device_id": device_ids,
    "timestamp": timestamps,
    "temperature": temperatures,
    "humidity": humidity,
    "vibration": vibration
})

chunk.to_csv(
    path,
    mode="w" if first_chunk else "a",
    header=first_chunk,
    index=False
)
first_chunk = False

# Reference aggregates for validating processing results
expected_temp_sum += np.bincount(
    device_ids,
    weights=temperatures,
    minlength=NUM_DEVICES
```

```
    )

    expected_counts += np.bincount(
        device_ids,
        minlength=NUM_DEVICES
    ).astype(np.int64)

    # FIXED: count hot readings without weighted bincount.
    # This returns integer counts and prevents UFuncTypeError.
    expected_hot_counts += np.bincount(
        device_ids[temperatures > TEMP_THRESHOLD],
        minlength=NUM_DEVICES
    ).astype(np.int64)

    written += n

    if written % 1_000_000 == 0 or written == n_records:
        print(f"  Generated {written:,} / {n_records:,} records")

expected_avg = np.divide(
    expected_temp_sum,
    expected_counts,
    out=np.zeros(NUM_DEVICES, dtype=np.float64),
    where=expected_counts != 0
)
```

```
    reference = {
        "expected_records": n_records,
        "device_counts": expected_counts,
        "average_temperature": expected_avg,
        "hot_counts": expected_hot_counts
    }

    return reference


# ---------------------- HADOOP-STYLE PROCESSING -----------------

def process_hadoop_style(path):
    """
    Simulate Hadoop-style batch aggregation using chunked processing
    and NumPy aggregation. This does not launch a real Hadoop cluster.
    """

    temp_sums = np.zeros(NUM_DEVICES, dtype=np.float64)
    device_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
    hot_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
    records_processed = 0

    for chunk in pd.read_csv(path, chunksize=CHUNK_SIZE):
        ids = chunk["device_id"].to_numpy(dtype=np.int64)
        temps = chunk["temperature"].to_numpy(dtype=np.float64)
```

```
    temp_sums += np.bincount(
        ids, weights=temps, minlength=NUM_DEVICES
    )
    device_counts += np.bincount(
        ids, minlength=NUM_DEVICES
    ).astype(np.int64)

    # Correct integer counting of readings above threshold
    hot_counts += np.bincount(
        ids[temps > TEMP_THRESHOLD],
        minlength=NUM_DEVICES
    ).astype(np.int64)

    records_processed += len(chunk)

avg_temp = np.divide(
    temp_sums,
    device_counts,
    out=np.zeros(NUM_DEVICES, dtype=np.float64),
    where=device_counts != 0
)

return avg_temp, hot_counts, records_processed
```

```
# ---------------------- SPARK-STYLE PROCESSING ------------------

def process_spark_style(path):
    """
    Simulate Spark-style group aggregation with Pandas.
    This is not actual Apache Spark or a distributed execution.
    """

    temp_sums = np.zeros(NUM_DEVICES, dtype=np.float64)
    device_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
    hot_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
    records_processed = 0

    for chunk in pd.read_csv(path, chunksize=CHUNK_SIZE):
        chunk["hot_reading"] = (
            chunk["temperature"] > TEMP_THRESHOLD
        ).astype(np.int64)

        grouped = chunk.groupby("device_id").agg(
            temperature_sum=("temperature", "sum"),
            reading_count=("temperature", "count"),
            hot_count=("hot_reading", "sum")
        )

        ids = grouped.index.to_numpy(dtype=np.int64)
```

```
        temp_sums[ids] += grouped[
            "temperature_sum"
        ].to_numpy(dtype=np.float64)

        device_counts[ids] += grouped[
            "reading_count"
        ].to_numpy(dtype=np.int64)

        hot_counts[ids] += grouped[
            "hot_count"
        ].to_numpy(dtype=np.int64)

        records_processed += len(chunk)

    avg_temp = np.divide(
        temp_sums,
        device_counts,
        out=np.zeros(NUM_DEVICES, dtype=np.float64),
        where=device_counts != 0
    )

    return avg_temp, hot_counts, records_processed


# ---------------------- MAIN EXPERIMENT -------------------------
```

```
ingestion_results = []
performance_results = []
scalability_results = []

for n_records in DATASET_SIZES:

    print("\n" + "=" * 70)
    print(f"DATASET SIZE: {n_records:,} RECORDS")
    print("=" * 70)

    csv_path = os.path.join(
        OUTPUT_DIR, f"ioe_sensor_data_{n_records}.csv"
    )

    # Generate dataset and reference results
    generation_start = time.perf_counter()
    reference = generate_dataset(n_records, csv_path)
    generation_time = time.perf_counter() - generation_start

    # Verify saved CSV row count
    verify_start = time.perf_counter()
    actual_records = 0

    for chunk in pd.read_csv(
        csv_path, usecols=["device_id"], chunksize=CHUNK_SIZE
    ):
```

```
    actual_records += len(chunk)

verification_time = time.perf_counter() - verify_start
records_match = actual_records == n_records

ingestion_results.append({
    "Dataset_Records": n_records,
    "Expected_Records": n_records,
    "Actual_Records": actual_records,
    "Record_Count_Match": records_match,
    "Generation_Time_s": round(generation_time, 3),
    "Verification_Time_s": round(verification_time, 3),
    "CSV_Size_MB": round(os.path.getsize(csv_path) / (1024**2), 2)
})

print(f"\nDataset generated: {csv_path}")
print(f"Expected records: {n_records:,}")
print(f"Actual CSV records: {actual_records:,}")
print(f"Record count verified: {records_match}")
print(f"Generation time: {generation_time:.2f} seconds")

# Run both approaches three times
hadoop_times = []
spark_times = []
hadoop_results = []
spark_results = []
```

```
for run in range(1, NUM_RUNS + 1):

    print(f"\nRun {run}/{NUM_RUNS}")

    start = time.perf_counter()
    h_result = process_hadoop_style(csv_path)
    h_elapsed = time.perf_counter() - start

    hadoop_times.append(h_elapsed)
    hadoop_results.append(h_result)
    print(f"  Hadoop-style processing: {h_elapsed:.2f} seconds")

    start = time.perf_counter()
    s_result = process_spark_style(csv_path)
    s_elapsed = time.perf_counter() - start

    spark_times.append(s_elapsed)
    spark_results.append(s_result)
    print(f"  Spark-style processing:  {s_elapsed:.2f} seconds")

avg_hadoop_time = float(np.mean(hadoop_times))
avg_spark_time = float(np.mean(spark_times))

# Validate results from the final run of each method
h_avg, h_hot, h_records = hadoop_results[-1]
```

```
s_avg, s_hot, s_records = spark_results[-1]


avg_temp_match = np.allclose(
    h_avg, s_avg, rtol=1e-7, atol=1e-7
) and np.allclose(
    h_avg, reference["average_temperature"],
    rtol=1e-7, atol=1e-7
)


hot_count_match = (
    np.array_equal(h_hot, s_hot)
    and np.array_equal(h_hot, reference["hot_counts"])
)


hadoop_record_match = (
    h_records == n_records
    and np.array_equal(
        np.bincount(
            np.arange(NUM_DEVICES),
            minlength=NUM_DEVICES
        )[:0],
        np.array([], dtype=np.int64)
    )
)


# Validate processed row totals and per-device counts
```

```
h_counts = np.zeros(NUM_DEVICES, dtype=np.int64)
for chunk in pd.read_csv(
    csv_path, chunksize=CHUNK_SIZE, usecols=["device_id"]
):
    h_counts += np.bincount(
        chunk["device_id"].to_numpy(dtype=np.int64),
        minlength=NUM_DEVICES
    ).astype(np.int64)

all_records_processed = (
    h_records == n_records
    and s_records == n_records
    and np.array_equal(h_counts, reference["device_counts"])
)

output_match = (
    avg_temp_match
    and hot_count_match
    and all_records_processed
    and records_match
)

hadoop_throughput = n_records / avg_hadoop_time
spark_throughput = n_records / avg_spark_time

speedup = (
```

```
    avg_hadoop_time / avg_spark_time
    if avg_spark_time > 0 else np.nan
)

improvement = (
    (avg_hadoop_time - avg_spark_time)
    / avg_hadoop_time * 100
    if avg_hadoop_time > 0 else np.nan
)

performance_results.append({
    "Dataset_Records": n_records,
    "Hadoop_Avg_Time_s": round(avg_hadoop_time, 3),
    "Spark_Avg_Time_s": round(avg_spark_time, 3),
    "Hadoop_Throughput_records_s": round(hadoop_throughput, 2),
    "Spark_Throughput_records_s": round(spark_throughput, 2),
    "Speedup_Hadoop_over_Spark": round(speedup, 3),
    "Spark_Time_Improvement_Percent": round(improvement, 2),
    "Average_Temperature_Match": avg_temp_match,
    "Hot_Count_Match": hot_count_match,
    "All_Records_Processed": all_records_processed,
    "Output_Match": output_match
})

scalability_results.append({
    "Dataset_Records": n_records,
```

```
        "Hadoop_Time_s": avg_hadoop_time,
        "Spark_Time_s": avg_spark_time,
        "Hadoop_Throughput": hadoop_throughput,
        "Spark_Throughput": spark_throughput
    })

    print("\nValidation:")
    print(f"  Average temperatures match: {avg_temp_match}")
    print(f"  Hot-reading counts match:   {hot_count_match}")
    print(f"  All records processed:      {all_records_processed}")
    print(f"  Final output match:         {output_match}")

    del reference
    gc.collect()


# ---------------------- RESULTS TABLES --------------------------

ingestion_df = pd.DataFrame(ingestion_results)
performance_df = pd.DataFrame(performance_results)
scalability_df = pd.DataFrame(scalability_results)

print("\n\n" + "=" * 70)
print("TABLE 1: DATA GENERATION AND RECORD-COUNT VERIFICATION")
print("=" * 70)
display(ingestion_df)
```

```
print("\n" + "=" * 70)
print("TABLE 2: HADOOP-STYLE VS SPARK-STYLE PERFORMANCE COMPARISON")
print("=" * 70)
display(performance_df)

print("\n" + "=" * 70)
print("TABLE 3: SCALABILITY RESULTS")
print("=" * 70)
display(scalability_df)



# ---------------------- GRAPHS ---------------------------------

sizes_millions = [
    n / 1_000_000 for n in performance_df["Dataset_Records"]
]

plt.figure(figsize=(9, 5))
plt.plot(
    sizes_millions, performance_df["Hadoop_Avg_Time_s"],
    marker="o", label="Hadoop-style"
)
plt.plot(
    sizes_millions, performance_df["Spark_Avg_Time_s"],
    marker="s", label="Spark-style"
```

```
)
plt.title("Execution Time vs Dataset Size")
plt.xlabel("Dataset Size (million records)")
plt.ylabel("Average Execution Time (seconds)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()


plt.figure(figsize=(9, 5))
plt.plot(
    sizes_millions,
    performance_df["Hadoop_Throughput_records_s"],
    marker="o", label="Hadoop-style"
)
plt.plot(
    sizes_millions,
    performance_df["Spark_Throughput_records_s"],
    marker="s", label="Spark-style"
)
plt.title("Processing Throughput vs Dataset Size")
plt.xlabel("Dataset Size (million records)")
plt.ylabel("Throughput (records/second)")
plt.grid(True)
plt.legend()
plt.tight_layout()
```

```
plt.show()


plt.figure(figsize=(9, 5))
plt.bar(
    [str(n) for n in performance_df["Dataset_Records"]],
    performance_df["Speedup_Hadoop_over_Spark"]
)
plt.axhline(1.0, linestyle="--", label="Equal execution time")
plt.title("Speedup: Hadoop-Style Time / Spark-Style Time")
plt.xlabel("Dataset Size (records)")
plt.ylabel("Speedup Ratio")
plt.grid(axis="y")
plt.legend()
plt.tight_layout()
plt.show()



# ---------------------- EXPORT RESULTS --------------------------


ingestion_file = os.path.join(
    OUTPUT_DIR, "ingestion_results.csv"
)
performance_file = os.path.join(
    OUTPUT_DIR, "performance_comparison.csv"
)
scalability_file = os.path.join(
```

```
    OUTPUT_DIR, "scalability_results.csv"
)

ingestion_df.to_csv(ingestion_file, index=False)
performance_df.to_csv(performance_file, index=False)
scalability_df.to_csv(scalability_file, index=False)

print("\n" + "=" * 70)
print("EXPERIMENT COMPLETED")
print("=" * 70)
print("Result files saved:")
print(ingestion_file)
print(performance_file)
print(scalability_file)

# Download the three result tables in Google Colab
try:
    from google.colab import files

    print("\nDownloading result CSV files...")
    files.download(ingestion_file)
    files.download(performance_file)
    files.download(scalability_file)

except ImportError:
    print("Not running in Google Colab; files remain in OUTPUT_DIR.")
```

```
# ---------------------- FORMULAS AND CONCLUSION ------------------

print("\nPERFORMANCE FORMULAS")
print("Throughput = Number of records / Execution time")
print("Speedup = Hadoop-style time / Spark-style time")
print("Time improvement (%) = (Hadoop time - Spark time)")
print("                    / Hadoop time × 100")

if performance_df["Output_Match"].all():
    print("\nVALIDATION RESULT: All output comparisons passed.")
else:
    print("\nVALIDATION RESULT: Some checks failed; inspect the tables.")

print(
    "\nIMPORTANT: These are local Python simulations of Hadoop-style "
    "and Spark-style processing. They do not execute real Hadoop, HDFS, "
    "or Apache Spark. The timing results compare two local processing "
    "approaches and must not be reported as actual Hadoop-vs-Spark "
    "cluster benchmarks."
)
```

```
======================================================================
IoE BIG DATA PROCESSING: HADOOP-STYLE vs SPARK-STYLE SIMULATION
======================================================================
Devices: 100
Dataset sizes: [1000000, 5000000, 10000000]
Runs per method: 3
Generating datasets in chunks to reduce memory usage...


======================================================================
DATASET SIZE: 1,000,000 RECORDS
======================================================================
  Generated 1,000,000 / 1,000,000 records

Dataset generated: /content/ioe_big_data_results/ioe_sensor_data_1000000.csv
Expected records: 1,000,000
Actual CSV records: 1,000,000
Record count verified: True
Generation time: 4.24 seconds

Run 1/3
  Hadoop-style processing: 0.49 seconds
  Spark-style processing:  0.59 seconds

Run 2/3
  Hadoop-style processing: 0.48 seconds
  Spark-style processing:  0.53 seconds

Run 3/3
  Hadoop-style processing: 0.47 seconds
  Spark-style processing:  0.60 seconds

Validation:
  Average temperatures match: True
  Hot-reading counts match:   True
  All records processed:      True
  Final output match:         True


======================================================================
DATASET SIZE: 5,000,000 RECORDS
======================================================================
  Generated 1,000,000 / 5,000,000 records
  Generated 2,000,000 / 5,000,000 records
  Generated 3,000,000 / 5,000,000 records
  Generated 4,000,000 / 5,000,000 records
  Generated 5,000,000 / 5,000,000 records

Dataset generated: /content/ioe_big_data_results/ioe_sensor_data_5000000.csv
Expected records: 5,000,000
Actual CSV records: 5,000,000
Record count verified: True
Generation time: 23.67 seconds

Run 1/3
  Hadoop-style processing: 2.99 seconds
  Spark-style processing:  3.57 seconds

Run 2/3
  Hadoop-style processing: 2.34 seconds
  Spark-style processing:  2.65 seconds

Run 3/3
  Hadoop-style processing: 2.32 seconds
  Spark-style processing:  3.51 seconds

Validation:
  Average temperatures match: True
  Hot-reading counts match:   True
  All records processed:      True
  Final output match:         True

======================================================================
DATASET SIZE: 10,000,000 RECORDS
======================================================================
  Generated 1,000,000 / 10,000,000 records
  Generated 2,000,000 / 10,000,000 records
  Generated 3,000,000 / 10,000,000 records
  Generated 4,000,000 / 10,000,000 records
```

```
Dataset generated: /content/ioe_big_data_results/ioe_sensor_data_10000000.csv
Expected records: 10,000,000
Actual CSV records: 10,000,000
Record count verified: True
Generation time: 47.65 seconds

Run 1/3
  Hadoop-style processing: 6.57 seconds
  Spark-style processing:  5.23 seconds

Run 2/3
  Hadoop-style processing: 6.01 seconds
  Spark-style processing:  5.60 seconds

Run 3/3
  Hadoop-style processing: 4.58 seconds
  Spark-style processing:  6.88 seconds

Validation:
  Average temperatures match: True
  Hot-reading counts match:   True
  All records processed:      True
  Final output match:         True


======================================================================
TABLE 1: DATA GENERATION AND RECORD-COUNT VERIFICATION
======================================================================
```

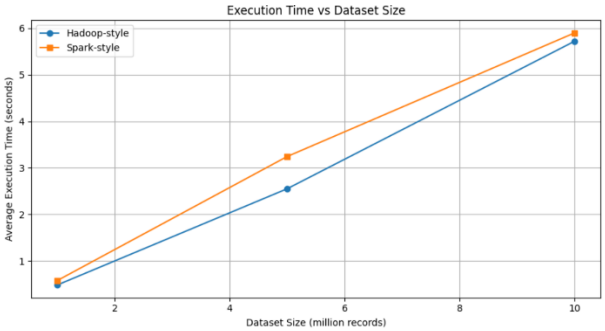
	Dataset_Records	Expected_Records	Actual_Records	Record_Count_Match	Generation_Time_s	Verification_Time_s	CSV_Size_MB
0	1000000	1000000	1000000	True	4.237	0.324	30.12
1	5000000	5000000	5000000	True	23.667	1.493	150.60
2	10000000	10000000	10000000	True	47.648	2.919	301.21

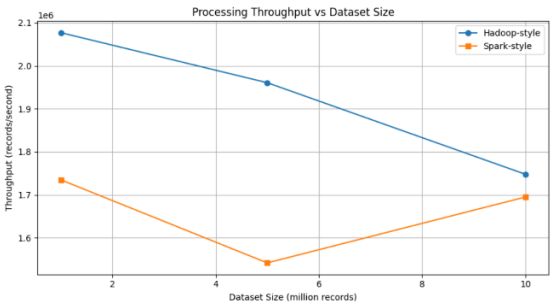
```
======================================================================
TABLE 2: HADOOP-STYLE VS SPARK-STYLE PERFORMANCE COMPARISON
======================================================================
```

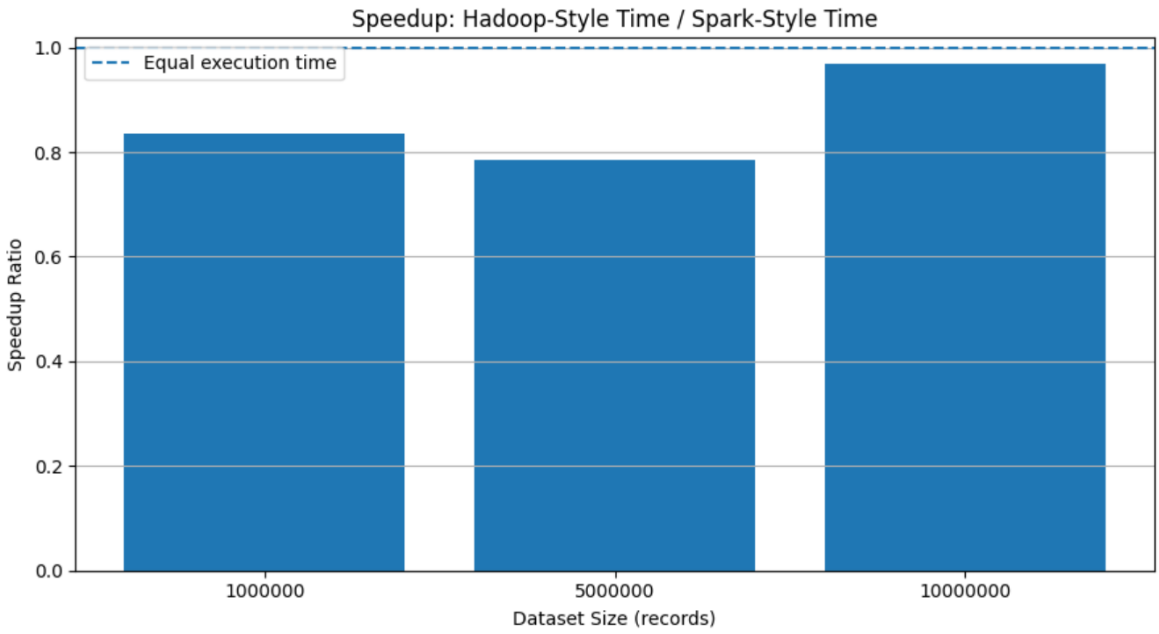
	Dataset_Records	Hadoop_Avg_Time_s	Spark_Avg_Time_s	Hadoop_Throughput_records_s	Spark_Throughput_records_s
0	1000000	0.481	0.576	2076873.50	1734806.66
1	5000000	2.550	3.243	1960558.81	1541751.93
2	10000000	5.721	5.900	1747799.58	1694788.21

```
======================================================================
TABLE 3: SCALABILITY RESULTS
======================================================================
```

	Dataset_Records	Hadoop_Time_s	Spark_Time_s	Hadoop_Throughput	Spark_Throughput
0	1000000	0.481493	0.576433	2.076873e+06	1.734807e+06
1	5000000	2.550293	3.243064	1.960559e+06	1.541752e+06
2	10000000	5.721480	5.900442	1.747800e+06	1.694788e+06






Speedup: Hadoop-Style Time / Spark-Style Time
Equal execution time
Speedup Ratio
1.0
0.8
0.6
0.4
0.2
0.0
1000000
5000000
10000000
Dataset Size (records)